CPAD Viva Prep Lab Assignment 03

Study Guide for Lab 03 — Adaptive & Responsive Design Theory, Code Walkthrough, and Viva Q&A

Repository: github.com/Chaitanya-666/CPADLABFlutterWidgets

Part A — Lab 03 Theory: Adaptive & Responsive Design

Experiment 3: Developing Adaptive and Responsive Designs in Flutter

Lab 03 required extending the Lab 02 Expense Tracker application so that its layout automatically adjusts to different screen sizes, orientations, and device types. The experiment builds on the `Row`, `Column`, and `Stack` structures from Lab 02 and introduces four new Flutter layout tools: `MediaQuery`, `LayoutBuilder`, `OrientationBuilder`, and the flexible widgets `Expanded`, `Flexible`, and `Wrap`.

Lab 03 — What was built

- **Adaptive switch:** `LayoutBuilder` at 600 dp breakpoint — mobile layout below, tablet two-pane layout above
- **Orientation-aware grid:** `OrientationBuilder` changes grid columns (2 portrait, 3 landscape)
- **Responsive stats:** `StatsRow` wraps to 2-per-line below 360 dp via `LayoutBuilder` + `Wrap`
- **Responsive chart:** `ChartSection` height reduces in landscape via `OrientationBuilder`
- **New files:** `responsive_expense_grid.dart`; modified: `home_screen.dart`, `stats_row.dart`, `chart_section.dart`
- **Build target:** Flutter Web (CanvasKit renderer), tested at mobile (414 dp), tablet portrait (800 dp), and tablet landscape (1024 dp)

Part B — Theory Deep Dive

Adaptive vs Responsive — The Core Distinction

Adaptive design means serving different layouts to different device classes. The app detects the device class (via screen width or platform) and renders a fundamentally different widget tree — for example, a bottom navigation bar on mobile vs a navigation rail on tablet, or a single-column list on mobile vs a two-pane master–detail on tablet. The key characteristic is that the layout *switches* at a breakpoint.

Responsive design means making a single layout fluidly adapt to any width. The same widget tree renders at every size, but flex factors (`Expanded`, `Flexible`), `Wrap`, and relative sizing ensure nothing overflows. The key characteristic is that the layout *stretches* without changing structure.

> **Analysis — How this app uses both**
>
> The `LayoutBuilder` branch at 600 dp is *adaptive* (two different widget trees). The `Expanded` cards in the `StatsRow`, the `Wrap` in the chart legend, and the `OrientationBuilder` grid column count are *responsive* (the same widget tree adapts). A well-built Flutter app always uses both: adaptive for large structural changes, responsive for fine-grained adaptation within each structure.

MediaQuery — The Screen-Level API

`MediaQuery.of(context)` returns a `MediaQueryData` object with:

Property	What it tells you
`size`	Full screen `Size(width, height)` in logical pixels
`orientation`	`Orientation.portrait` or `Orientation.landscape`
`devicePixelRatio`	Physical pixels per logical pixel (2.0 on retina, 3.0 on some Android)
`textScaleFactor`	User's font size preference (accessibility)
`padding`	Safe area insets (notches, status bar)
`viewInsets`	Keyboard height when visible

In this experiment, `MediaQuery.of(context).orientation` is used in the tablet layout to decide the left pane width (380 dp portrait, 320 dp landscape).

> **Observation — MediaQuery gives screen size, not parent size**
>
> `MediaQuery.size` always returns the *full screen* dimensions, even if your widget is inside a narrow pane. If you need the space your widget has been given by its parent, use `LayoutBuilder` instead. This matters in desktop apps where windows can be resized: a `MediaQuery`-based breakpoint would only change when the screen changes (never on desktop), while a `LayoutBuilder`-based breakpoint adapts correctly to window resizing.

LayoutBuilder — Parent-Constraint-Aware Building

`LayoutBuilder` takes a `builder` callback that receives `BoxConstraints` — the min/max width and height the parent has allocated. You inspect `constraints.maxWidth` and return a different widget subtree based on the available width.

```
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth >= 600) {
      return _tabletLayout(...);
    }
    return _mobileLayout(...);
  },
)
```

Key Principle — LayoutBuilder is preferred over MediaQuery for layout decisions

`LayoutBuilder` respects the parent's constraints, making the widget reusable in any context — full-screen, split-view pane, or dialog. `MediaQuery` is for device-level properties (orientation, pixel ratio) that don't depend on the widget's position in the tree.

OrientationBuilder — Automatic Rebuild on Rotation

`OrientationBuilder` takes a `builder` callback that receives the current `Orientation` enum. It registers a dependency on `MediaQuery` and automatically rebuilds whenever the orientation changes.

```
OrientationBuilder(
  builder: (context, orientation) {
    final crossAxisCount =
        orientation == Orientation.portrait ? 2 : 3;
    return GridView.builder(
      gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: crossAxisCount,
      ),
      ...
    );
  },
)
```

Viva Tip — OrientationBuilder vs MediaQuery.orientation

Both detect orientation. But `MediaQuery.of(context).orientation` in a `StatelessWidget.build()` does *not* trigger a rebuild on rotation by itself — you need an `OrientationBuilder` or a `StatefulWidget` with a `WidgetsBindingObserver`. `OrientationBuilder` handles this automatically.

Expanded, Flexible, Wrap — The Flexible Trio

Widget	Behaviour	Used in
`Expanded(flex: 1)`	Forces child to fill its flex allocation (min = max)	Tablet Row right pane, StatsRow cards
`Flexible(fit: loose)`	Gives child up to its flex allocation, allows shrinking	Not used here — Expanded is sufficient

`Wrap`	Moves children to next line when horizontal space runs out	Chart legend, StatsRow sub-360dp fallback

Breakpoints — The 600 dp Convention

Width	Device class	Layout in this app
< 360 dp	Small phone	StatsRow wraps to 2-per-line
360–599 dp	Phone	Single-column mobile scroll view
600–839 dp	Tablet portrait	Two-pane: 380 dp summary + 2-col grid
840–1199 dp	Tablet landscape	Two-pane: 320 dp summary + 3-col grid
≥ 1200 dp	Desktop / web	Same as tablet landscape (could add more columns)

Part C — Code Walkthrough: File by File

1. `home_screen.dart` — The LayoutBuilder Switch

The `HomeScreen` widget's body is now a `LayoutBuilder` that checks `constraints.maxWidth >= 600`. Below 600 dp, it calls `_mobileLayout()` — the same Lab 02 single-column `CustomScrollView`. At 600 dp or above, it calls `_tabletLayout()`, which returns a `Row` with a fixed-width `SizedBox` (left pane) and an `Expanded` (right pane). The right pane contains an `OrientationBuilder` that sets `crossAxisCount` to 2 (portrait) or 3 (landscape).

```dart
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:provider/provider.dart';
import '../providers/expense_provider.dart';
import '../widgets/chart_section.dart';
import '../widgets/expense_card.dart';
import '../widgets/stats_row.dart';
import '../widgets/responsive_expense_grid.dart';
import 'add_expense_screen.dart';

/// Lab 03 — Adaptive & Responsive Design
///
/// The HomeScreen now wraps its body in a [LayoutBuilder] to detect the
/// available width and switches between a single-column **mobile layout**
/// (< 600 dp) and a two-column **tablet/web layout** (>= 600 dp).
///
/// Inside the tablet layout, an [OrientationBuilder] wraps the expense
/// list section so that the grid column count changes between portrait
/// (2 columns) and landscape (3 columns).
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  /// Material Design breakpoint — widely used convention.
```

```
  static const double tabletBreakpoint = 600.0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Expense Tracker'),
        centerTitle: true,
        elevation: 0,
      ),
      body: Consumer<ExpenseProvider>(
        builder: (context, provider, child) {
          if (!provider.isLoaded) {
            return const Center(child: CircularProgressIndicator());
          }

          final expenses = provider.expenses;
          final total = provider.totalExpenses;
          final categoryTotals = provider.categoryTotals;
          final currencyFormat =
              NumberFormat.currency(symbol: '₹', decimalDigits: 0);

          // ── LayoutBuilder: detect available width ──────────────
          // This is the core of Lab 03. Instead of assuming a mobile
          // width, we read constraints.maxWidth and branch.
          return LayoutBuilder(
            builder: (context, constraints) {
              final isTablet = constraints.maxWidth >= tabletBreakpoint;

              if (isTablet) {
                return _tabletLayout(
                  context: context,
                  expenses: expenses,
                  total: total,
                  categoryTotals: categoryTotals,
                  currencyFormat: currencyFormat,
                  provider: provider,
                  constraints: constraints,
                );
              }
              return _mobileLayout(
                context: context,
                expenses: expenses,
                total: total,
                categoryTotals: categoryTotals,
                currencyFormat: currencyFormat,
                provider: provider,
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton.extended(
```

```
        onPressed: () => Navigator.push(
          context,
          MaterialPageRoute(builder: (_) => const AddExpenseScreen()),
        ),
        icon: const Icon(Icons.add),
        label: const Text('Add Expense'),
      ),
    );
  }

  // ─────────────────────────────────────────────────────────────────
  //  MOBILE LAYOUT  (< 600 dp)
  //  Single-column scroll view — identical to Lab 02 but now explicitly
  //  selected by the LayoutBuilder branch above.
  // ─────────────────────────────────────────────────────────────────
  Widget _mobileLayout({
    required BuildContext context,
    required List expenses,
    required double total,
    required Map<String, double> categoryTotals,
    required NumberFormat currencyFormat,
    required ExpenseProvider provider,
  }) {
    return CustomScrollView(
      slivers: [
        SliverToBoxAdapter(child: _totalCard(context, total, expenses.length,
currencyFormat)),
        SliverToBoxAdapter(
          child: Padding(
            padding: const EdgeInsets.only(bottom: 8),
            child: StatsRow(expenses: expenses),
          ),
        ),
        SliverToBoxAdapter(
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16),
            child: Text('Breakdown',
                style:
Theme.of(context).textTheme.titleLarge?.copyWith(fontWeight: FontWeight.bold)),
          ),
        ),
        SliverToBoxAdapter(child: ChartSection(categoryTotals: categoryTotals,
total: total)),
        SliverToBoxAdapter(
          child: Padding(
            padding: const EdgeInsets.fromLTRB(16, 24, 16, 8),
            child: Text('Recent Transactions',
                style:
Theme.of(context).textTheme.titleLarge?.copyWith(fontWeight: FontWeight.bold)),
          ),
        ),
        if (expenses.isEmpty)
          const SliverFillRemaining(
```

```
            child: Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Icon(Icons.receipt_long, size: 64, color: Colors.grey),
                  SizedBox(height: 16),
                  Text('No expenses yet. Tap + to add one.'),
                ],
              ),
            ),
          )
        else
          SliverList(
            delegate: SliverChildBuilderDelegate(
              (context, index) {
                final expense = expenses[index];
                final catTotal = categoryTotals[expense.category] ?? 0;
                final pct = total > 0 ? (catTotal / total) * 100 : 0.0;
                return ExpenseCard(
                  expense: expense,
                  categoryPercentage: pct,
                  onTap: () => Navigator.push(
                    context,
                    MaterialPageRoute(builder: (_) => AddExpenseScreen(expense:
expense)),
                  ),
                  onDelete: () => provider.deleteExpense(expense.id),
                );
              },
              childCount: expenses.length,
            ),
          ),
        const SliverPadding(padding: EdgeInsets.only(bottom: 80)),
      ],
    );
  }

  // ─────────────────────────────────────────────────────────────────────
  //  TABLET / WEB LAYOUT  (>= 600 dp)
  //  Two-pane master–detail: left pane holds the summary (total card,
  //  stats, chart) in a fixed-width column; right pane holds the
  //  expense list in a responsive grid whose column count depends on
  //  orientation (via OrientationBuilder).
  // ─────────────────────────────────────────────────────────────────────
  Widget _tabletLayout({
    required BuildContext context,
    required List expenses,
    required double total,
    required Map<String, double> categoryTotals,
    required NumberFormat currencyFormat,
    required ExpenseProvider provider,
    required BoxConstraints constraints,
  }) {
```

```
    // MediaQuery: read orientation for the overall pane split ratio
    final isLandscape =
        MediaQuery.of(context).orientation == Orientation.landscape;

    return Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // ── Left pane: summary (fixed width 380 on portrait, 320 on landscape)
        SizedBox(
          width: isLandscape ? 320 : 380,
          child: CustomScrollView(
            slivers: [
              SliverToBoxAdapter(child: _totalCard(context, total,
expenses.length, currencyFormat)),
              SliverToBoxAdapter(
                child: Padding(
                  padding: const EdgeInsets.only(bottom: 8),
                  child: StatsRow(expenses: expenses),
                ),
              ),
              SliverToBoxAdapter(
                child: Padding(
                  padding: const EdgeInsets.symmetric(horizontal: 16),
                  child: Text('Breakdown',
                      style:
Theme.of(context).textTheme.titleLarge?.copyWith(fontWeight: FontWeight.bold)),
                ),
              ),
              SliverToBoxAdapter(child: ChartSection(categoryTotals:
categoryTotals, total: total)),
              const SliverPadding(padding: EdgeInsets.only(bottom: 80)),
            ],
          ),
        ),
        // ── Right pane: expense grid
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Padding(
                padding: const EdgeInsets.fromLTRB(16, 16, 16, 8),
                child: Text('Recent Transactions',
                    style:
Theme.of(context).textTheme.titleLarge?.copyWith(fontWeight: FontWeight.bold)),
              ),
              Expanded(
                // ── OrientationBuilder: change grid columns on rotation
                child: OrientationBuilder(
                  builder: (context, orientation) {
                    // Portrait → 2 columns, Landscape → 3 columns
                    final crossAxisCount =
                        orientation == Orientation.portrait ? 2 : 3;
                    if (expenses.isEmpty) {
```

```
                        return Center(
                          child: Column(
                            mainAxisAlignment: MainAxisAlignment.center,
                            children: [
                              const Icon(Icons.receipt_long, size: 64, color:
Colors.grey),
                              const SizedBox(height: 16),
                              const Text('No expenses yet. Tap + to add one.'),
                            ],
                          ),
                        );
                      }
                      return ResponsiveExpenseGrid(
                        expenses: expenses,
                        categoryTotals: categoryTotals,
                        total: total,
                        crossAxisCount: crossAxisCount,
                        onTap: (expense) => Navigator.push(
                          context,
                          MaterialPageRoute(builder: (_) =>
AddExpenseScreen(expense: expense)),
                        ),
                        onDelete: (id) => provider.deleteExpense(id),
                      );
                    },
                  ),
                ),
              ],
            ),
          ),
        ],
      );
    }

    // ── Shared total-balance gradient card ──────────────────────────
    Widget _totalCard(BuildContext context, double total, int count, NumberFormat
fmt) {
      return Container(
        margin: const EdgeInsets.all(16),
        padding: const EdgeInsets.all(20),
        decoration: BoxDecoration(
          gradient: LinearGradient(
            colors: [
              Theme.of(context).colorScheme.primary,
              Theme.of(context).colorScheme.primaryContainer,
            ],
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
          ),
          borderRadius: BorderRadius.circular(20),
          boxShadow: [
            BoxShadow(
              color: Theme.of(context).colorScheme.primary.withValues(alpha: 0.3),
```

```
            blurRadius: 10,
            offset: const Offset(0, 4),
          ),
        ],
      ),
      child: Column(
        children: [
          const Text('Total Expenses',
              style: TextStyle(fontSize: 14, color: Colors.white70)),
          const SizedBox(height: 8),
          Text(fmt.format(total),
              style: const TextStyle(
                  fontSize: 36, fontWeight: FontWeight.bold, color:
Colors.white)),
          const SizedBox(height: 4),
          Text('$count transaction${count != 1 ? 's' : ''}',
              style: const TextStyle(color: Colors.white70)),
        ],
      ),
    );
  }
}
```

2. `responsive_expense_grid.dart` — The GridView

A `GridView.builder` with `SliverGridDelegateWithFixedCrossAxisCount`. The `crossAxisCount` is passed in from the `OrientationBuilder`. Each grid cell is an `ExpenseCard` — the same widget used in the mobile list, ensuring visual consistency across breakpoints.

```dart
import 'package:flutter/material.dart';
import '../models/expense.dart';
import 'expense_card.dart';

/// Lab 03 — Responsive expense grid for the tablet/web layout.
///
/// Uses [GridView.builder] inside an [OrientationBuilder] (the
/// crossAxisCount is passed in by the caller, who read it from
/// OrientationBuilder).  Each grid cell is an [ExpenseCard] wrapped
/// in a [Card] with a percentage badge — the same visual as the mobile
/// list, but laid out in a grid.
///
/// The [Wrap] widget inside the legend below the chart (see
/// ChartSection) is the other responsive container in this app:
/// it flows category legend chips onto multiple lines on narrow
/// screens.
class ResponsiveExpenseGrid extends StatelessWidget {
  final List<Expense> expenses;
  final Map<String, double> categoryTotals;
  final double total;
  final int crossAxisCount;
  final void Function(Expense)? onTap;
```

```
  final void Function(String)? onDelete;

  const ResponsiveExpenseGrid({
    super.key,
    required this.expenses,
    required this.categoryTotals,
    required this.total,
    required this.crossAxisCount,
    this.onTap,
    this.onDelete,
  });

  @override
  Widget build(BuildContext context) {
    return GridView.builder(
      padding: const EdgeInsets.fromLTRB(16, 0, 16, 80),
      gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: crossAxisCount,
        childAspectRatio: 2.8,
        crossAxisSpacing: 8,
        mainAxisSpacing: 8,
      ),
      itemCount: expenses.length,
      itemBuilder: (context, index) {
        final expense = expenses[index];
        final catTotal = categoryTotals[expense.category] ?? 0;
        final pct = total > 0 ? (catTotal / total) * 100 : 0.0;
        return ExpenseCard(
          expense: expense,
          categoryPercentage: pct,
          onTap: onTap != null ? () => onTap!(expense) : null,
          onDelete: onDelete != null ? () => onDelete!(expense.id) : null,
        );
      },
    );
  }
}
```

3. `stats_row.dart` — Wrap as Responsive Fallback

The `StatsRow` now wraps its `Row` in a `LayoutBuilder`. Below 360 dp, it switches from `Row` (four `Expanded` children) to `Wrap` (two `SizedBox` children per line, each `(maxWidth - 8) / 2` wide). This prevents the four cards from being too cramped on very narrow screens.

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import '../models/expense.dart';

/// Quick-stats strip showing Today / Week / Month / Average spend.
///
/// Lab 02 base: `Row` → 4 × `Expanded` → `Column`.
```

```
///
/// Lab 03 enhancement: On screens narrower than 360 dp the four
/// `Expanded` cards become too cramped.  A [LayoutBuilder] wraps the
/// row; when `constraints.maxWidth < 360` we switch from a single `Row`
/// to a [Wrap] that lets cards flow onto a second line (2 per row).
/// This demonstrates the responsive use of `Wrap` alongside `Expanded`.
class StatsRow extends StatelessWidget {
  final List<Expense> expenses;

  const StatsRow({super.key, required this.expenses});

  @override
  Widget build(BuildContext context) {
    final fmt = NumberFormat.compactCurrency(symbol: '₹', decimalDigits: 0);

    final now = DateTime.now();
    final todayStart = DateTime(now.year, now.month, now.day);
    final weekStart = todayStart.subtract(const Duration(days: 6));
    final monthStart = DateTime(now.year, now.month, 1);

    double todayTotal = 0;
    double weekTotal = 0;
    double monthTotal = 0;
    for (final e in expenses) {
      if (e.date.isAfter(todayStart.subtract(const Duration(seconds: 1)))) {
        todayTotal += e.amount;
      }
      if (e.date.isAfter(weekStart.subtract(const Duration(seconds: 1)))) {
        weekTotal += e.amount;
      }
      if (e.date.isAfter(monthStart.subtract(const Duration(seconds: 1)))) {
        monthTotal += e.amount;
      }
    }
    final avg = expenses.isEmpty
        ? 0.0
        : expenses.fold(0.0, (s, e) => s + e.amount) / expenses.length;

    final cards = [
      _StatCard(label: 'Today', value: fmt.format(todayTotal), accent:
Colors.orange),
      _StatCard(label: 'This Week', value: fmt.format(weekTotal), accent:
Colors.blue),
      _StatCard(label: 'This Month', value: fmt.format(monthTotal), accent:
Colors.green),
      _StatCard(label: 'Avg / Txn', value: fmt.format(avg), accent:
Colors.purple),
    ];

    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16),
      // ── LayoutBuilder: choose Row (>= 360 dp) or Wrap (< 360 dp)
      child: LayoutBuilder(
```

```
        builder: (context, constraints) {
          if (constraints.maxWidth < 360) {
            // Wrap: cards flow to next line — responsive fallback
            return Wrap(
              spacing: 8,
              runSpacing: 8,
              children: cards
                  .map((c) => SizedBox(
                        width: (constraints.maxWidth - 8) / 2,
                        child: c,
                      ))
                  .toList(),
            );
          }
          // Row → 4 × Expanded (default, Lab 02 behaviour)
          return Row(
            children: [
              for (int i = 0; i < cards.length; i++) ...[
                Expanded(child: cards[i]),
                if (i < cards.length - 1) const SizedBox(width: 8),
              ],
            ],
          );
        },
      ),
    );
  }
}

/// A single stat card. A Column of label + value inside a Card.
class _StatCard extends StatelessWidget {
  final String label;
  final String value;
  final Color accent;

  const _StatCard({
    required this.label,
    required this.value,
    required this.accent,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 1,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      child: Padding(
        padding: const EdgeInsets.symmetric(vertical: 12, horizontal: 8),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(label,
                style: TextStyle(
```

```
                    fontSize: 11,
                    color: Colors.grey.shade600,
                    fontWeight: FontWeight.w500)),
            const SizedBox(height: 4),
            Text(value,
                style: TextStyle(
                    fontSize: 15, fontWeight: FontWeight.bold, color: accent)),
          ],
        ),
      ),
    );
  }
}
```

4. `chart_section.dart` — OrientationBuilder for Chart Height

The `ChartSection` wraps its `PieChart` in an `OrientationBuilder` that reduces the chart height from 200 px to 160 px in landscape. The `Wrap` legend (from Lab 02) already handles flowing category chips onto multiple lines.

```dart
import 'package:flutter/material.dart';
import 'package:fl_chart/fl_chart.dart';
import '../models/expense.dart';

/// Pie chart section with category legend.
///
/// Lab 03 enhancement: The legend below the pie chart already uses a
/// [Wrap] widget (added in Lab 02) so that category chips flow onto
/// multiple lines on narrow screens.  Additionally, an
/// [OrientationBuilder] wraps the chart so that in landscape the
/// chart height is reduced (160 instead of 200) to leave more room
/// for the grid beside it.
class ChartSection extends StatelessWidget {
  final Map<String, double> categoryTotals;
  final double total;

  const ChartSection({
    super.key,
    required this.categoryTotals,
    required this.total,
  });

  @override
  Widget build(BuildContext context) {
    if (total == 0) {
      return const SizedBox(
        height: 200,
        child: Center(child: Text('No expenses yet')),
      );
    }
```

```
    final sections = categoryTotals.entries.where((e) => e.value > 0).map((e) {
      final percentage = (e.value / total) * 100;
      return PieChartSectionData(
        color: Color(Category.colors[e.key] ?? 0xFF90A4AE),
        value: e.value,
        title: '${percentage.toStringAsFixed(1)}%',
        radius: 60,
        titleStyle: const TextStyle(
          fontSize: 12,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
      );
    }).toList();

    // ── OrientationBuilder: shrink chart height in landscape
    return OrientationBuilder(
      builder: (context, orientation) {
        final chartHeight = orientation == Orientation.landscape ? 160.0 : 200.0;
        return Column(
          children: [
            SizedBox(
              height: chartHeight,
              child: PieChart(
                PieChartData(
                  sections: sections,
                  centerSpaceRadius: 40,
                  sectionsSpace: 2,
                ),
              ),
            ),
            const SizedBox(height: 16),
            // Wrap: legend chips flow onto multiple lines on narrow screens
            Wrap(
              spacing: 16,
              runSpacing: 8,
              children: categoryTotals.entries.where((e) => e.value > 0).map((e)
{
                return Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Container(
                      width: 12,
                      height: 12,
                      decoration: BoxDecoration(
                        color: Color(Category.colors[e.key] ?? 0xFF90A4AE),
                        borderRadius: BorderRadius.circular(3),
                      ),
                    ),
                    const SizedBox(width: 6),
                    Text(e.key),
                  ],
                );
```

```
              }).toList(),
            ),
          ],
        );
      },
    );
  }
}
```

Part D — Experiment Timeline

Step 1: Wrap the Scaffold body in LayoutBuilder

The first change was wrapping the `HomeScreen`'s `Consumer<ExpenseProvider>` body in a `LayoutBuilder`. The `builder` callback checks `constraints.maxWidth >= 600` and branches. Initially, both branches returned the same mobile layout to verify the `LayoutBuilder` was wired correctly.

Step 2: Build the tablet two-pane layout

The `_tabletLayout()` method was written: a `Row` with a `SizedBox(width: 380)` left pane (summary: total card, StatsRow, ChartSection in a `CustomScrollView`) and an `Expanded` right pane (Recent Transactions header + expense list). Initially, the right pane used the same `SliverList` as the mobile layout.

Step 3: Add OrientationBuilder for grid column count

The `SliverList` in the right pane was replaced with an `OrientationBuilder` wrapping a `GridView.builder`. The `crossAxisCount` is set to 2 in portrait and 3 in landscape. A new `ResponsiveExpenseGrid` widget was extracted to keep the code clean.

Step 4: Add Wrap to StatsRow for narrow screens

The `StatsRow` was wrapped in a `LayoutBuilder` with a sub-breakpoint at 360 dp. Below 360 dp, the `Row` of four `Expanded` cards switches to a `Wrap` of `SizedBox` cards, 2 per line.

Step 5: Add OrientationBuilder to ChartSection

The `ChartSection`'s `PieChart` was wrapped in an `OrientationBuilder` that reduces the chart height from 200 to 160 px in landscape, leaving more room for the grid.

Step 6: Test at three viewport sizes and capture screenshots

The compiled Flutter web build was served over HTTP and captured at mobile (414 × 960), tablet portrait (800 × 1024), and tablet landscape (1024 × 768) viewport sizes. The layout adapted correctly at all three sizes with no overflow, clipping, or overlap.

Part E — Viva Questions and Answers

Category 1: Adaptive vs Responsive

Q1. What is the difference between adaptive design and responsive design?

Adaptive design serves different layouts to different device classes — the layout switches at a breakpoint (e.g., single-column on mobile, two-pane on tablet). Responsive design uses one layout that fluidly adapts to any width via flex factors and `Wrap`. Flutter apps use both: `LayoutBuilder` for the adaptive switch, `Expanded`/`Wrap` for responsive fluidity.

Q2. Give an example of an adaptive design decision in your app.

The `LayoutBuilder` at 600 dp that switches between `_mobileLayout()` (single-column `CustomScrollView`) and `_tabletLayout()` (two-pane `Row` with summary + grid). This is adaptive because the widget tree is fundamentally different on each side of the breakpoint.

Q3. Give an example of a responsive design decision in your app.

The `StatsRow`'s `Wrap` fallback below 360 dp, or the `ChartSection`'s `Wrap` legend. These are responsive because the same widget tree is used at all widths — only the flow of children changes.

Q4. Can a single app be both adaptive and responsive?

Yes, and it should be. This app is adaptive (LayoutBuilder switches layouts at 600 dp) and responsive (Expanded cards resize, Wrap legends reflow, OrientationBuilder changes grid columns). Adaptive handles large structural changes; responsive handles fine-grained adaptation within each structure.

Category 2: MediaQuery and LayoutBuilder

Q5. What is MediaQuery and what properties does it provide?

`MediaQuery.of(context)` returns `MediaQueryData` with `size`, `orientation`, `devicePixelRatio`, `textScaleFactor`, `padding`, and `viewInsets`. It reads the full screen's properties. In this app, `MediaQuery.of(context).orientation` is used to decide the tablet left pane width.

Q6. What is LayoutBuilder and how does it differ from MediaQuery?

`LayoutBuilder` reads the `BoxConstraints` passed down by the parent widget — the space the widget has been given, which may be less than the full screen. `MediaQuery` reads the full screen size. Use `LayoutBuilder` for layout decisions based on available width; use `MediaQuery` for device-level properties.

Q7. Why is LayoutBuilder preferred over MediaQuery for layout decisions?

Because `LayoutBuilder` respects the parent's constraints, making the widget reusable in any context — full-screen, split-view pane, or dialog. A `MediaQuery`-based breakpoint would only change when the screen changes, not when the window or pane is resized.

Q8. What breakpoint did you use and why?

600 dp, following Material Design guidelines. Below 600 dp, a single-column layout is the right choice. Above 600 dp, there is room for a two-pane layout. An additional sub-breakpoint at 360 dp switches the StatsRow from Row to Wrap.

Category 3: OrientationBuilder

Q9. What does OrientationBuilder do?

It rebuilds its child whenever the device orientation changes. Its `builder` receives `(BuildContext, Orientation)`. In this app, it changes the expense grid from 2 columns (portrait) to 3 (landscape).

Q10. How is OrientationBuilder different from checking MediaQuery.orientation?

`MediaQuery.of(context).orientation` in a `StatelessWidget.build()` does not trigger a rebuild on rotation by itself. `OrientationBuilder` registers a dependency on `MediaQuery` and rebuilds automatically. Use `OrientationBuilder` when the widget structure must change on rotation.

Q11. What happens to the grid when you rotate from portrait to landscape?

The `OrientationBuilder` detects the change and sets `crossAxisCount` from 2 to 3. The left pane width shrinks from 380 to 320 dp. The chart height reduces from 200 to 160 px. The stats row wraps to 2-per-line in the narrower left pane.

Category 4: Expanded, Flexible, Wrap

Q12. What does Expanded do?

`Expanded` wraps a child to make it fill its allocated flex space inside a `Row` or `Column`. It sets the child's min and max width equal to the flex allocation. In this app, `Expanded` is used in the tablet Row to give the right pane all remaining width after the fixed left pane.

Q13. What is the difference between Expanded and Flexible?

`Expanded` forces the child to fill its flex space (min = max). `Flexible(fit: FlexFit.loose)` gives the child up to its flex space but allows it to be smaller. Use `Expanded` when the child should always fill its share; use `Flexible` when it should shrink-wrap but be bounded.

Q14. What does Wrap do, and where is it used in your app?

`Wrap` arranges children in a sequence, automatically moving them to the next line when there is not enough space. In this app, `Wrap` is used in the `ChartSection` legend (category chips flow onto multiple lines) and in the `StatsRow` (stat cards wrap 2-per-line below 360 dp).

Q15. Why use Wrap instead of a GridView for the legend?

The legend has a small, fixed number of items (5–7 categories) and each item is small (a coloured dot + text). `Wrap` is simpler than `GridView` for this use case and handles the flow naturally. `GridView` is better for large, uniform datasets where lazy building matters.

Category 5: Design Decisions and Gotchas

Q16. Why did you use a Row with SizedBox + Expanded for the tablet layout?

The left pane needs a fixed width (380 dp) so the summary (card, stats, chart) is always the same size. The right pane needs to fill all remaining space. `SizedBox(width: 380)` fixes the left pane; `Expanded` gives the right pane a flex of 1, taking all remaining width. Without `Expanded`, the right pane would size to its intrinsic content width.

Q17. What happens if the screen is between 600 and 839 dp?

The `LayoutBuilder` selects the tablet layout. The `OrientationBuilder` detects portrait and sets the grid to 2 columns. The left pane is 380 dp, leaving 420–459 dp for the right pane — enough for 2 cards at 200 dp each.

Q18. How do you ensure the app doesn't overflow at any screen size?

Three mechanisms: (1) `LayoutBuilder` selects the correct layout for each width; (2) `Expanded` (a specialised `Flexible` with `FlexFit.tight`) ensures children share space proportionally; (3) `Wrap` ensures legends and stats flow onto multiple lines rather than overflowing. Additionally, `SingleChildScrollView` in the form prevents clipping.

Q19. What is the difference between constraints.maxWidth and MediaQuery.size.width?

`constraints.maxWidth` is the width the parent has given to this widget — it may be less than the screen width (e.g., inside a pane). `MediaQuery.size.width` is always the full screen width. `LayoutBuilder` gives you `constraints.maxWidth`; `MediaQuery` gives you the screen size.

Q20. Did you need to change the data model or state management for Lab 03?

No. The `Expense` model, `ExpenseProvider`, and `StorageService` are unchanged from Lab 01. Lab 03 is purely a layout/UI change — the `LayoutBuilder`, `OrientationBuilder`, and `Wrap` are all in the presentation layer. The data layer is untouched.

Part F — GitHub Repository & How to Run

Repository

The complete source code for all three labs is published at:

github.com/Chaitanya-666/CPADLABFlutterWidgets

How to Clone and Run

```
git clone https://github.com/Chaitanya-666/CPADLABFlutterWidgets.git
```

```
cd CPADLABFlutterWidgets
```

```
flutter pub get
```

```
flutter run -d chrome
```

To test the responsive behaviour, resize the browser window from wide to narrow and observe the layout switch at 600 dp. Rotate the device (or use Chrome DevTools' device toolbar) to see the grid column count change.

How to Recompile the PDFs

```
cd typst/
```

```
typst compile CpadLabAssignment03ChaitanyaShinde231070066.typ
```

```
typst compile CPAD_VivaPrep_Lab03_ChaitanyaShinde231070066.typ
```

(Requires typst 0.13+. The only warning is "unknown font: linux libertine" which is non-fatal.)

END OF ASSIGNMENT 03

CHAITANYA SHINDE | Final Year BTECH CE 231070066